SG-ALNS: Segment-Level Graph Reinforcement Learning to Configure ALNS for the VRPTW

Duong Q. Vo
Faculty of Information Technology
Vinh University
Nghe An, Vietnam
235748020110196@vinhuni.edu.vn

Sang X. Tran*
Institute of Information Technology
and Digital Transformation, Vinh University
Nghe An, Vietnam
sangtx@vinhuni.edu.vn

Uyen T. Nguyen*
Faculty of Information Technology
Vinh University
Nghe An, Vietnam
uyennt@vinhuni.edu.vn

***Abstract*—Reinforcement learning (RL) is increasingly used to control Adaptive Large Neighbourhood Search (ALNS), and its gains are usually credited to state-dependent control. We examine this claim on the vehicle routing problem with time windows (VRPTW) with SG-ALNS, a segment-level graph RL framework designed to be favourable to RL. The policy configures ALNS once per segment of iterations, so that PPO can be trained on complete search runs. Each decision jointly selects the operators, the destroy scale and the annealing temperature from a graph state with dynamic time-window features, and the action space includes the classical roulette-wheel rule. SG-ALNS outperforms our re-implementations of iteration-level RL agents trained on short episodes, and its pre-designated training seed uses fewer vehicles than the roulette wheel (418.0 vs. 421.4 on Solomon; a mean of 419.0 over 3 seeds). Six controls (policy inspection, training-seed replication, tuned baselines, a dynamic-headroom search, equal run time, and transfer across budgets and sizes), however, show that these gains stem from learning an effective configuration rather than from state-dependent control: 11 of 12 policies collapse to fixed actions and ignore counterfactual changes of their state, cheaply tuned static configurations are statistically equivalent in fleet size, and the advantage vanishes at equal run time. These findings offer practical guidance for evaluating learning-guided metaheuristics, within the scope of the VRPTW setting studied.**

***Index Terms*—reinforcement learning, adaptive large neighbourhood search, algorithm configuration, evaluation methodology, vehicle routing problem with time windows**

I. Introduction

The vehicle routing problem with time windows (VRPTW) asks for minimum-cost routes that serve every customer within its time window. It is NP-hard, and Adaptive Large Neighbourhood Search (ALNS) [1], [2] is one of the most widely used metaheuristics for it [3]. At every iteration, a *destroy* operator removes q customers from the incumbent, a *repair* operator reinserts them, and a simulated-annealing (SA) criterion decides acceptance; a roulette wheel (RW) biases operator selection towards recently successful operators. A meta-analysis [4], however, found that this adaptive layer improves solution quality by only about 0.14% on average. This makes ALNS a natural target for learned *algorithm control* [5], also known as parameter control [6] or dynamic algorithm configuration (DAC) [7], [8]. Recent works train deep reinforcement learning (RL) agents to control ALNS at every iteration [9]–[12] and report gains over RW.

These agents share three limitations. First, to keep the Markov decision process (MDP) tractable, they are trained on short episodes (20 decisions in GRLOS [11], 100 iterations in DR-ALNS [10]) or on runs shorter than at deployment (1,000 vs. up to 10,000 iterations in DRLH [9]), and Johnn et al. [11] report degradation outside the training horizon. Second, apart from DR-ALNS [10], they fix the destroy scale and the temperature, although both strongly affect ALNS [1], [13]. Third, to our knowledge, none of them can fall back to the RW rule as one of its actions.

More fundamentally, it is rarely checked *why* the agent wins: does it exploit the search state, or has it merely found a better static configuration? Control subsumes configuration, so a controller can win without being adaptive; DAC studies therefore compare learned policies with the best static configuration [7], [14]. In RL-controlled ALNS, Reijnen et al. [10] compare with an ALNS tuned by SMAC, but whether the learned policies actually depend on the state, and how they vary across training runs, has not been reported.

We study this question with SG-ALNS (Segment-level Graph ALNS), a controller built to give RL the best possible chance. Following the DAC formulation, in which a policy reconfigures the algorithm at fixed intervals [7], one decision configures the next K iterations, so that proximal policy optimisation (PPO) can be trained on complete ALNS runs. Each decision jointly selects the operators, the destroy scale and the temperature from a graph state with dynamic time-window features, and the action space contains the RW rule. Our contributions are:

- **A segment-level MDP for ALNS** on the VRPTW that combines interval-based decisions trained on full trajectories, joint control of operators, destroy scale and temperature, a dynamic time-window graph state, a telescoping hierarchical reward and an *expert-inclusive* action space. Each ingredient has precedents (Sect. II), but their combination is new.
- **An evaluation protocol with six controls** for RL-controlled ALNS. Static baselines and Friedman–Nemenyi tests are established practice in DAC and algorithm benchmarking [14], [15]; we combine them

*Corresponding authors: Sang X. Tran (sangtx@vinhuni.edu.vn) and Uyen T. Nguyen (uyennt@vinhuni.edu.vn).

TABLE I
POSITIONING AGAINST RL-CONTROLLED ALNS WORKS (✓: YES, –: NO, AS REPORTED IN THE CITED PAPERS).

Work	Decision	Graph	Dyn. TW	Learns q	Learns τ	Full-run training	RW action
DRLH [9]	iteration	–	–	–	–	✓‡	–
DR-ALNS [10]	iteration	–	–	✓	✓	–†	–
GRLOS [11]	iteration	✓	–	–	–	–	–
SG-ALNS (ours)	segment	✓	✓	✓	✓	✓	✓

† 100-iteration episodes, evaluated on 100–200 iterations.
‡ complete 1,000-iteration runs, deployed for up to 10,000 iterations.

with a policy-collapse diagnostic, training-seed replication, tuned static *and* tuned ALNS-RW baselines with a tuning budget smaller than one training run, a dynamic-headroom test, a time-matched comparison, and transfer across budgets and instance sizes up to $n = 400$.

- **Evidence that learned ALNS control on the VRPTW is, in our setting, learned configuration**: 11 of the 12 trained policies are state-independent and the exception is worse, seed-to-seed differences dominate, tuned configurations match the policy, time-dependent schedules offer only small headroom that the policies do not exploit, and transfer follows the training distribution rather than any adaptation.

II. RELATED WORK

ALNS and its adaptive layer. ALNS [1], [2] combines a portfolio of destroy operators (random, worst, Shaw [16] and string [17] removals), repair operators (greedy and regret insertion), an acceptance criterion [13], and a roulette wheel that adapts operator weights to their recent success. Unlike neural LNS [18], [19], which learns repair operators, we keep classical operators and learn how to schedule them.

RL-controlled ALNS. Kallestad et al. [9] (DRLH) replace the adaptive layer with a deep RL hyper-heuristic that selects a heuristic at every iteration; it is trained with PPO on complete 1,000-iteration runs under a fixed SA schedule. DR-ALNS [10], the closest work to ours, also sets the destroy severity and the SA temperature at every iteration, from seven flat features and 100-iteration training episodes; joint control of operators and parameters is therefore not new. GRLOS [11] encodes the solution with a graph neural network (GNN) and selects operators with a deep Q-network, using a fixed destroy scale and short training horizons. PPO-ALNS [12] integrates PPO-based operator selection into ALNS for the VRPTW. As summarised in Table I, SG-ALNS differs from these works by segment-level decisions, a graph state with dynamic time-window features, and an action space that contains the RW rule.

Configuration versus control. Offline configurators such as irace [20] fix parameter values for an instance distribution, whereas parameter control [6] and DAC [7], [8], often solved by RL [21], adapt them during a run. We bring this question to RL-controlled ALNS and add what, to our knowledge, has not been reported there: inspection of the learned actions, replication over training seeds, equal-time comparisons, and transfer across instance sizes.

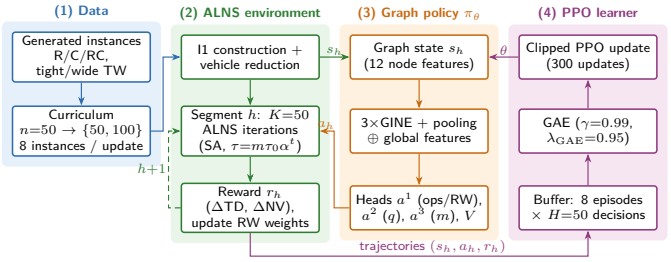


Fig. 1. Training framework of SG-ALNS: generated instances (1) feed complete ALNS episodes split into segments of $K = 50$ iterations (2); before each segment the graph policy maps the state s_h to an action a_h (3); PPO updates θ from 8 parallel episodes (4). Algorithm 1 details one episode.

III. PROBLEM AND METHOD

a) VRPTW.: Customers $i \in \{1, \dots, n\}$ have demand d_i, service time σ_i and a hard time window $[e_i, l_i]$; identical vehicles of capacity Q start and end at depot 0, and travel times equal unrounded Euclidean distances c_{ij}. Service at i starts at $b_i = \max\{e_i,\, b_p + \sigma_p + c_{pi}\} \le l_i$, where p is the predecessor of i. As is customary, we minimise the number of vehicles NV, then the total distance TD, via $f = 10^4\,\mathrm{NV} + \mathrm{TD}$; all solutions are re-checked by an independent feasibility checker.

b) Underlying ALNS.: All methods share the same ALNS: Solomon's I1 construction [22], a vehicle-reduction phase of $10n$ iterations [1], and a main phase of T destroy–repair iterations with SA acceptance (τ_0 accepts a 5% worse solution with probability 0.5; $\tau_t = \tau_0\alpha^t$ with $\alpha = 0.01^{1/T}$). The portfolio has six destroy operators (random, worst, Shaw, string, route and waiting-time) and three repair operators (greedy, regret-3 and noisy greedy). Insertion feasibility is checked in $O(1)$ per position with forward time slack [23], in Numba kernels. After each segment (100 iterations in ALNS-RW, K inside SG-ALNS), RW updates the weight of operator o as $w_o \leftarrow (1-\rho)w_o + \rho\,\psi_o/u_o$, where ψ_o is the total score of o in the segment (33, 9 or 13 per use) and u_o its number of uses, with $\rho = 0.1$; the destroy scale is drawn as $q \sim \mathcal{U}[0.05n, 0.30n]$.

c) Segment-level MDP.: An episode is one complete main phase, split into $H = T/K$ segments ($K = 50$). At the start of each segment, the agent observes the state s_h and selects a factorised action $a_h = (a^1, a^2, a^3)$. The component $a^1 \in \{0, \dots, 18\}$ selects either a *lead* destroy–repair pair, used with probability 0.7 while the other operators share the remaining probability uniformly, or ($a^1 = 18$) the current RW distribution, whose weights keep being updated. The component a^2 selects $q = \kappa n$ with $\kappa \in \{5, 10, 20, 30\}\%$ or the default range $\mathcal{U}[5, 30]\%$, and a^3 selects a deviation $m \in \{0.3, 1, 3\}$ *around* the cooling schedule, $\tau = m\tau_0\alpha^t$. The action $(18, \mathrm{default}, 1)$ reproduces ALNS-RW; we call this action space *expert-inclusive*, in the spirit of DAC policies that select among existing heuristics [21]. A headroom study motivates it: no fixed lead pair or destroy scale dominates RW (Table S4), and a first design without the RW action barely

Algorithm 1: SG-ALNS (one episode)

Input: instance I, policy π_θ, segment length K, budget T
Output: best solution $x^\star$; trajectory $(s_h, a_h, r_h)_{h=1}^{H}$ (training only)

$x \leftarrow \text{VehicleReduction}(\text{SolomonI1}(I))$; $x^\star \leftarrow x$; $\boldsymbol{w} \leftarrow \mathbf{1}$; $t \leftarrow 0$;
for $h = 1, \dots, H$ **do** `// one decision per segment`
  $s_h \leftarrow \text{GraphState}(x, x^\star, t/T, \tau/\tau_0,$ operator success rates$)$;
  $a_h = (a^1, a^2, a^3) \sim \pi_\theta(\cdot \mid s_h)$ (training) or $\arg\max \pi_\theta(\cdot \mid s_h)$ (deployment);
  $(p_D, p_R) \leftarrow \text{RW}(\boldsymbol{w})$ if $a^1 = 18$, else lead pair a^1 (prob. 0.7);
  $\mathcal{Q} \leftarrow$ destroy range of a^2; $m \leftarrow m(a^3)$;
  for $k = 1, \dots, K$ **do**
    $\tau \leftarrow m\,\tau_0 \alpha^t$; $o_D \sim p_D$, $o_R \sim p_R$, $q \sim \mathcal{Q}$; $x' \leftarrow o_R(o_D(x, q))$; $t \leftarrow t + 1$;
    if $f(x') < f(x^\star)$ **then** $x^\star \leftarrow x'$;
    if $\text{AcceptSA}(x', x, \tau)$ **then** $x \leftarrow x'$;
  $r_h \leftarrow$ Eq. (1); update $\boldsymbol{w}$ from the operators' scores in the segment;
return $x^\star$ and $(s_h, a_h, r_h)_{h=1}^{H}$;

improved on it. The reward

$$r_h = 100\,\frac{\text{TD}^\star_{h-1} - \text{TD}^\star_h}{\text{TD}^\star_0} + \lambda\,(\text{NV}^\star_{h-1} - \text{NV}^\star_h), \qquad \lambda = 5, \tag{1}$$

uses the best solution $x^\star$ after each segment; since consecutive terms cancel, the return of an episode equals the final hierarchical improvement.

d) State and policy (Fig. 1, stage 3).: The state graph contains the 10 nearest-neighbour arcs of each node and the arcs of the incumbent. Each node has 6 static features (coordinates, demand, time window and service time) and 6 dynamic ones (waiting time, slack, forward time slack, load ratio, route position and a routed flag). A global vector holds the progress t/T, the temperature ratio τ/τ_0, a stagnation counter, the gaps of the current and initial solutions to the best one, the fleet ratio and the operators' success rates in the last segment. Three residual graph isomorphism layers with edge features (GINE) [24] of width 64, followed by mean and max pooling, feed three categorical heads and a value head (58k parameters). Output biases are initialised to favour the expert action, so that learning starts near ALNS-RW.

e) Training (Fig. 1, stages 1, 2 and 4).: Each of 300 PPO [25] updates runs Algorithm 1 with $T = 2{,}500$ on 8 generated instances in parallel and uses generalised advantage estimation (GAE) [26] ($\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, clip 0.2, Adam with learning rate $3 \cdot 10^{-4}$, entropy coefficient annealed from 0.01 to 0.001). Instances follow Solomon's design, with $n = 50$ for the first 40% of the updates and $n \in \{50, 100\}$ afterwards. No benchmark instance is used for training or model selection; we evaluate the final checkpoint of three training seeds (seed 0, used for all ablations, was designated beforehand). One training run takes 21–23 minutes on 8 cores.

f) Context-aware variant (SG-ALNS-C).: SG-ALNS observes neither n nor T, so it cannot condition q on the instance size. SG-ALNS-C appends $n/200$ and $T/5{,}000$ to the global vector and is trained on $n \in \{50, 100, 200\}$ with $T \in \{1{,}000, 2{,}500, 5{,}000\}$ drawn per update (33 minutes), a setting that favours instance-dependent behaviour.

IV. Experimental Setup

a) Benchmarks.: We use the 56 Solomon instances ($n = 100$) [22] and the 60+60 Gehring–Homberger instances with $n = 200$ and $n = 400$ [27]. For Solomon, the reference is the hierarchical best-known solution (BKS) table of SINTEF, cross-checked against the CVRPLIB solutions (identical on all 19 instances with the BKS fleet; none of our runs dominates a BKS). For Gehring–Homberger, hierarchical BKS are not available in machine-readable form (CVRPLIB lists distance-only solutions), so the reference is the *best solution found* over all our runs; these gaps are not comparable with BKS gaps reported elsewhere.

b) Baselines.: ALNS-Uni samples operators uniformly; ALNS-RW is the classical roulette wheel of Sect. III. **Tuned ALNS-RW** is the best of 36 RW configurations (update period, reaction factor ρ, maximum destroy scale $q_{\max}$ and temperature deviation m; selected: K=100, ρ=0.3, $q_{\max}$=40%, m=1). **Tuned static** is the best of 30 constant actions of the SG-ALNS action space (selected: String/Regret-3, $q = 30\%$, $m = 0.3$). Both are selected on 24 *generated* instances (3 seeds, $T = 5{,}000$) by the score $5\Delta\text{NV} + 100\Delta\text{TD}/\text{TD}$ against ALNS-RW (the weighting of Eq. (1)), in 3–6 CPU-minutes. The **iteration-level baselines** make one PPO decision per iteration over the 18 operator pairs. They are trained on 100-iteration episodes, as in DR-ALNS [10], with the same operators and rewards, and use either an MLP encoder (PPO-ALNS-like [12], 2,400 episodes) or our GNN encoder (GRLOS-like [11], PPO instead of DQN; stopped after 1,368 episodes, i.e. 57 minutes or 2.6× the training time of SG-ALNS, because per-iteration inference is much slower). Longer episodes would cost 10× more; DR-ALNS (another problem and budget) was not re-implemented.

c) Protocol.: Solomon runs use 5 seeds per instance (3 for the iteration-level baselines and the budget study), and Gehring–Homberger runs use 3. We report cumulative vehicles (CNV) and distance (CTD), the TD gap to the reference over runs that reach the reference NV, and the primal integral (PI) of f over the main phase, in % of the reference. Pairwise results are written W/T/L (p): the numbers of instances on which SG-ALNS has a lower, equal or higher mean f, and the p-value of a one-sided Wilcoxon signed-rank test (Holm-corrected within a table). Multiple comparisons use the Friedman test with Nemenyi critical differences [15] over the six methods run with 5 seeds (excluding the 3-seed iteration-level baselines). Each run uses one core of an Apple M1 (times include vehicle reduction and inference). Tables S1–S12 and Figs. S1–S4 are in the supplement. The tuned baselines, seeds 1–2, SG-ALNS-C, the time-matched study and the $n = 400$ set were added after we observed policy collapse; none uses benchmark data for selection. Code, trained models, raw results and the BKS table are available at https://github.com/XXXX/sg-alns.

TABLE II

SG-ALNS AGAINST BASELINES ON SOLOMON ($T = 5{,}000$, 5 SEEDS; *3 SEEDS, COMPARED ON COMMON SEEDS). GAP: TD GAP TO THE BKS OVER RUNS WITH THE BKS FLEET (THEIR NUMBER, OUT OF 280 OR *168). W/T/L: INSTANCES WHERE SG-ALNS HAS A LOWER/EQUAL/HIGHER MEAN f; p: TWO-SIDED SIGN TEST ON W VS. L, HOLM-CORRECTED. *OUR RE-IMPLEMENTATIONS, TRAINED ON 100-ITERATION EPISODES; BOTH ALWAYS CHOOSE WORST/REGRET-3 AND RETURN IDENTICAL SOLUTIONS.

Method	CNV	CTD	Gap % (runs)	PI %	Time s	W/T/L	p
BKS	405	57,187	0.00	–	–	–	–
SG-ALNS (seed 0, pre-designated)	418.0	57,188	1.48 (215)	5.05	2.15	–	–
SG-ALNS, 3 training seeds (mean±sd)	419.0±1.1	57,310	1.88±0.35	5.08	2.29	–	–
ALNS-RW	421.4	57,115	1.86 (201)	5.58	1.09	25/16/15	0.462
Tuned ALNS-RW	419.8	57,144	1.61 (206)	5.30	1.44	25/17/14	0.433
ALNS-Uni	421.0	57,352	2.09 (201)	5.44	1.10	35/16/5	<0.001
Tuned static	418.6	57,190	1.69 (212)	5.07	0.97	25/16/15	0.462
SG-ALNS-C	422.0	57,512	2.44 (195)	5.56	1.71	34/16/6	<0.001
SG-ALNS (sampled)	418.4	57,190	1.53 (213)	4.95	1.44	21/16/19	0.875
Iteration-level GNN*	425.3	57,424	2.63 (111)	5.83	18.80	34/15/7	<0.001
Iteration-level MLP*	425.3	57,424	2.63 (111)	5.83	10.66	34/15/7	<0.001

V. RESULTS

A. Main Results on Solomon

Table II reports $T = 5{,}000$. The pre-designated seed of SG-ALNS uses 418.0 vehicles against 421.4 for ALNS-RW (BKS: 405), with the lowest gap (1.48%) and primal integral, and the improvement is significant (25/16/15, $p_{\text{Holm}} = 0.011$); averaged over three training seeds it uses 419.0±1.1 vehicles (Sect. V-B). The 16 ties are C1/C2 instances solved identically by all methods; the gain is in vehicles on R1, R2 and RC1 (Fig. S3a). SG-ALNS pulls ahead of all baselines after about 2,500 iterations, the end of its training horizon (Fig. 2a). Both iteration-level agents are worse than SG-ALNS and even than ALNS-RW (425.3 vehicles), 5–9× slower, and stall after 1,000 iterations. Trained on 100-iteration episodes, they always apply Worst/Regret-3 (Fig. 3a), a choice that suits the start of the search but not 5,000 iterations. This train–deploy mismatch, avoided by segment-level training, is the one advantage of SG-ALNS that survives all controls below. Against the two *tuned* baselines, however, SG-ALNS is not significantly better (tuned ALNS-RW: $p = 0.065$; tuned static: $p = 0.253$). Over the six methods run with 5 seeds, the Friedman test rejects equal performance ($p = 1.1 \times 10^{-10}$), but SG-ALNS (mean rank 2.71), tuned static (2.96), tuned ALNS-RW (3.29) and ALNS-RW (3.32) form one group under the Nemenyi critical difference of 1.01 (Fig. S4a).

B. Learned Control or Learned Configuration?

a) Collapse.: In Fig. 3a, 11 of the 12 trained policies choose *one* action in 100% of their decisions on Solomon, on every instance. This includes SG-ALNS-C, which observes n and T yet selects the same action at $n = 100$, 200 and 400 and at every budget (Table S11). The only exception, without a^1, alternates between the default destroy range and $q = 20\%$ (8.7 switches per run) and is significantly *worse* than SG-ALNS ($p = 0.002$). Entropy collapses during training (Fig. S4b), and sampling actions instead of taking the argmax does not change the result ($p = 0.353$).

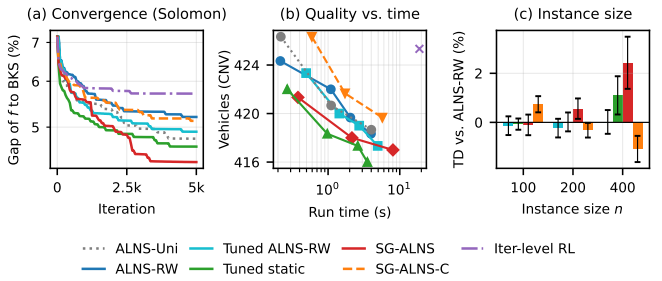


Fig. 2. (a) Mean gap of the best-so-far f to the BKS during the main phase (Solomon, 3 seeds; f is dominated by the fleet size). (b) Cumulative vehicles on Solomon vs. mean run time for $T \in \{1, 5, 10, 20\} \cdot 10^3$ (3 seeds); points further down-left are better. (c) Mean relative distance to ALNS-RW on instances where both use the same mean fleet (95% confidence intervals) for $n = 100$ (Solomon), 200 and 400 (Gehring–Homberger).

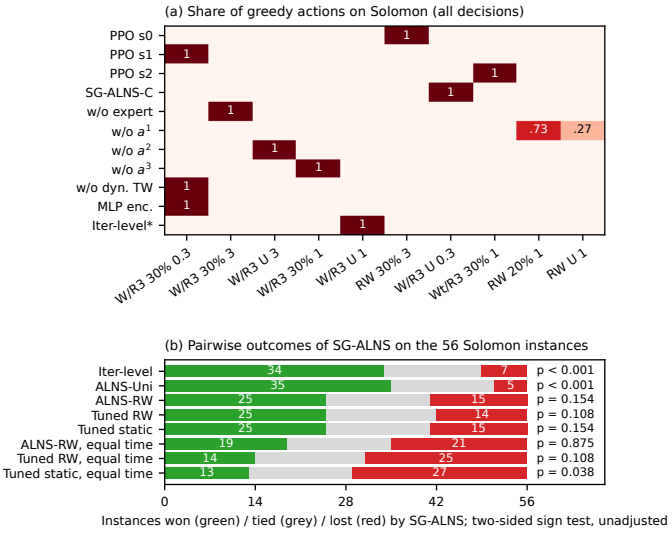


Fig. 3. (a) Share of each greedy action over all decisions on Solomon for the 12 trained policies (s0–s2: training seeds; the two iteration-level agents share a row). Actions are written lead pair, q, m with W = Worst, Wt = Wait, R3 = Regret-3, U = $\mathcal{U}[5, 30]$%. (b) Instance-wise score $5\Delta\text{NV} + 100\Delta\text{TD}/\text{TD}$ against ALNS-RW (diamond: mean). (c) Wins/ties/losses of SG-ALNS on the 56 instances, with unadjusted one-sided Wilcoxon p; "eq. t.": baselines run with $T = 10{,}000$ (equal time).

b) Training seeds.: If the policy encodes a constant, different training runs may find different ones, and they do: the three seeds learn three actions that share only $q = 30\%$ (Fig. 3a), and only seed 0 beats ALNS-RW significantly. In Fig. 3b, the score distributions of seeds 1 and 2 are centred near zero, and that of tuned static is as favourable as that of seed 0. Across seeds, SG-ALNS uses 419.0±1.1 vehicles with a gap of 1.88±0.35%. Likewise, the variants without a^1, a^2, dynamic features or message passing are significantly worse ($p \approx 0.002$), but each is another training run that converged to another constant (Fig. 3a), so these ablations say nothing about the value of the encoder or the features (Table S7).

c) Tuned configurations.: If the controller is a constant, a constant tuned offline should do as well, and it does: tuned static reaches 418.6 vehicles at less than half the run

time. Tuned ALNS-RW closes most of the gap with a larger maximum destroy scale ($q_{\max} = 40\%$) and a faster reaction ($\rho = 0.3$), moving ALNS-RW from 421.4 to 419.8 vehicles.

d) Is there headroom for control?: A collapse may reflect a weak learner rather than an absence of useful control, so we searched directly for time-dependent control: all 100 two-phase schedules (one action per half of the main phase) over the 10 best constant actions, with the tuning protocol of the tuned baselines (Table S6). The best schedule (String/Regret-3, 20%, m=0.3 $\rightarrow$ String/Regret-3, 30%, m=3) scores 0.81 against 0.70 for the best constant, which is not significant ($p = 0.51$); only 11 of the 90 non-constant schedules beat the best constant, and a schedule selected on two seeds scores less than the constant on the third (0.20 vs. 0.30). On Solomon the schedule does win more instances than the constant (W/T/L of the schedule: 26/15/15 (0.013), unadjusted), through shorter routes (gap 1.47% vs. 1.69%) at slightly more vehicles (419.0 vs. 418.6). Some time-dependent headroom thus exists (a smaller destroy scale early, a larger one and a higher temperature late), but it is small and fragile, and none of our policies exploits it. PPO hyper-parameters do not appear to cause the collapse: an additional policy, not among the 12 of Fig. 3a, trained with a constant entropy coefficient of 0.01 and neutral initial biases again uses 2 actions that differ only in q (0.78 of decisions on the first) and is worse than ALNS-RW (422.6 vehicles; its W/T/L against ALNS-RW: 10/16/30 (0.999)).

C. Equal Time, Budgets and Instance Size

a) Time-matched comparison.: SG-ALNS needs 2.15 s per run against 1.09 s for ALNS-RW. Inference takes only 3–4% of it (Fig. S3b); the rest is due to the large destroy scale. At equal run time, tuned static lies at or below SG-ALNS except at the shortest budget (Fig. 2b). Giving the baselines $T = 10{,}000$ roughly equalises run time; SG-ALNS then no longer beats ALNS-RW (19/16/21 (0.425)), and tuned ALNS-RW (418.8 vehicles, gap 1.25%) and tuned static (417.8, 1.28%) win on more instances than they lose (Fig. 3c; Table S9).

b) Budgets.: At $T = 1{,}000$, the budget closest to its training budget, SG-ALNS beats both tuned baselines (vs. tuned ALNS-RW: 25/14/17 (0.002); vs. tuned static: 27/14/15 (0.028)). At $T = 5{,}000$ the differences are not significant, and at $T = 20{,}000$ SG-ALNS loses more instances than it wins against tuned static (16/16/24 (0.753)): each constant is best near the budget it was optimised for (Table S10).

c) Instance size.: Fig. 2c transfers all settings unchanged to $n = 200$ and 400 (Table S12). The fixed $q = 30\%$ learned by SG-ALNS becomes harmful at $n = 400$: its distance is 2.4% above ALNS-RW, its gap to the best found solution grows to 5.01%, and it loses to ALNS-RW on most instances. SG-ALNS-C, trained on $n \leq 200$, is the best method at $n = 400$ (gap 1.53% vs. 2.79% for ALNS-RW; 43/0/17 (<0.001)) and ties with tuned ALNS-RW at $n = 200$, yet it is the weakest learned method on Solomon (422.0 vehicles). As its action is identical at all sizes, this is not adaptation: its constant (a random destroy range, low temperature) suits the larger instances of its training distribution but is too greedy for the tight Solomon instances.

D. Discussion

Segment-level RL reliably finds *good configurations* without hand-tuning and can find the best configuration we tested when trained on the deployment distribution (SG-ALNS-C at $n = 400$). In our setting, however, the learned policies are (almost) state-independent, and a direct search finds only small, fragile headroom for time-dependent schedules, consistent with [4]. This does not contradict the gains of [9]–[12], obtained in other settings, but shows which controls are needed before attributing them to adaptation. *Threats to validity:* our conclusions concern the VRPTW, our operators and action space; the tuned baselines use small grids rather than irace [20], which favours SG-ALNS; and the iteration-level baselines are re-implementations within our framework.

VI. Conclusion

We evaluated SG-ALNS, a segment-level RL controller for ALNS on the VRPTW, with rarely applied controls. It clearly outperforms iteration-level RL agents and, for its main training seed, the classical roulette wheel. Yet the policies are (almost) state-independent, vary across training seeds, are matched by cheap tuned configurations and lose their edge at equal run time. In our setting, learned ALNS "control" is, in effect, learned configuration; richer action spaces may leave more room for genuine control. Practitioners can obtain most of the benefit from cheap offline configuration, or from RL used as a configurator on the target distribution. Future work will test continuous, size-relative actions and electric VRPs.

Disclosure of Interests

The authors have no competing interests to declare that are relevant to the content of this article.

References

[1] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[2] D. Pisinger and S. Ropke, "A general heuristic for vehicle routing problems," *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.

[3] S. T. W. Mara, R. Norcahyo, P. Jodiawan, L. Lusiantoro, and A. P. Rifai, "A survey of adaptive large neighborhood search algorithms and applications," *Computers & Operations Research*, vol. 146, p. 105903, 2022.

[4] R. Turkeš, K. Sörensen, and L. M. Hvattum, "Meta-analysis of meta-heuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search," *European Journal of Operational Research*, vol. 292, no. 2, pp. 423–442, 2021.

[5] M. Karimi-Mamaghan, M. Mohammadi, P. Meyer, A. M. Karimi-Mamaghan, and E.-G. Talbi, "Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art," *European Journal of Operational Research*, vol. 296, no. 2, pp. 393–422, 2022.

[6] A. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 2, pp. 124–141, 1999.

[7] A. Biedenkapp, H. F. Bozkurt, T. Eimer, F. Hutter, and M. Lindauer, "Dynamic algorithm configuration: Foundation of a new meta-algorithmic framework," in *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 325. IOS Press, 2020, pp. 427–434.

[8] S. Adriaensen, A. Biedenkapp, G. Shala, N. Awad, T. Eimer, M. Lindauer, and F. Hutter, "Automated dynamic algorithm configuration," *Journal of Artificial Intelligence Research*, vol. 75, pp. 1633–1699, 2022.

[9] J. Kallestad, R. Hasibi, A. Hemmati, and K. Sörensen, "A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems," *European Journal of Operational Research*, vol. 309, no. 1, pp. 446–468, 2023.

[10] R. Reijnen, Y. Zhang, H. C. Lau, and Z. Bukhsh, "Online control of adaptive large neighborhood search using deep reinforcement learning," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 34, 2024, pp. 475–483.

[11] S.-N. Johnn, V.-A. Darvariu, J. Handl, and J. Kalcsics, "A graph reinforcement learning framework for neural adaptive large neighbourhood search," *Computers & Operations Research*, vol. 172, p. 106791, 2024.

[12] Z. Wang, Q. Cui, B. Tan, X. Yang, W. Zhou, and X. Huang, "Reinforcement learning-guided adaptive large neighborhood search for vehicle routing problem with time windows," *Journal of Combinatorial Optimization*, vol. 50, p. 35, 2025.

[13] A. Santini, S. Ropke, and L. M. Hvattum, "A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic," *Journal of Heuristics*, vol. 24, no. 5, pp. 783–815, 2018.

[14] T. Eimer, A. Biedenkapp, M. Reimer, S. Adriaensen, F. Hutter, and M. Lindauer, "DACBench: A benchmark library for dynamic algorithm configuration," in *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, 2021, pp. 1668–1674.

[15] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[16] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Principles and Practice of Constraint Programming – CP98*, ser. LNCS, vol. 1520. Springer, 1998, pp. 417–431.

[17] J. Christiaens and G. Vanden Berghe, "Slack induction by string removals for vehicle routing problems," *Transportation Science*, vol. 54, no. 2, pp. 417–433, 2020.

[18] A. Hottung and K. Tierney, "Neural large neighborhood search for the capacitated vehicle routing problem," in *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI)*, 2020, pp. 443–450.

[19] J. K. Falkner, D. Thyssens, and L. Schmidt-Thieme, "Large neighborhood search based on neural construction heuristics," *arXiv preprint arXiv:2205.00772*, 2022.

[20] M. López-Ibáñez, J. Dubois-Lacoste, L. P. Cáceres, M. Birattari, and T. Stützle, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.

[21] D. Speck, A. Biedenkapp, F. Hutter, R. Mattmüller, and M. Lindauer, "Learning heuristic selection with dynamic algorithm configuration," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 31, 2021, pp. 597–605.

[22] M. M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.

[23] M. W. P. Savelsbergh, "The vehicle routing problem with time windows: Minimizing route duration," *ORSA Journal on Computing*, vol. 4, no. 2, pp. 146–154, 1992.

[24] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.

[25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[26] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations (ICLR)*, 2016.

[27] H. Gehring and J. Homberger, "A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows," in *Proceedings of EUROGEN99*, 1999, pp. 57–64.